

TECHNICAL TRAINING COURSE

Web Services for Java EE



Comprehensive Course Materials

Topics Covered: JAX-WS · JAXB · SAAJ · JAX-RS · JAXR

Web Services Metadata · Provider/Dispatch APIs

Duration: 2 Days (16 hours)

Level: Intermediate – Java EE Developer / Architect

Version: 1.0

Table of Contents

Table of Contents	2
1. Scenarios for Hosting Web Services	5
1.1 Learning Objectives	5
1.2 Why Host Web Services in Java EE?	5
1.3 Common Hosting Scenarios	5
Scenario 1 — Business-to-Business (B2B) Integration	5
Scenario 2 — Internal Service Decomposition (SOA)	5
Scenario 3 — Legacy System Wrapping	5
Scenario 4 — RESTful Resource Exposure	6
1.4 Deployment Topologies	6
1.5 Container-Provided Services	6
1.6 Review Questions	6
2. Web Services in Java EE	7
2.1 Learning Objectives	7
2.2 The Java EE Web Services Stack	7
2.3 JSR 109 — Web Services for Java EE	7
2.4 Two Endpoint Types	8
2.5 Web Service Client Models	8
2.6 Review Questions	8
3. JAX-WS and JAXB	9
3.1 Learning Objectives	9
3.2 JAX-WS Overview	9
3.3 A Minimal JAX-WS Service	9
3.4 Key JAX-WS Annotations	9
3.5 JAXB — Java Architecture for XML Binding	10
JAXB Annotations	10
3.6 Marshalling and Unmarshalling	11
3.7 Review Questions	11
4. Web Services Metadata (JSR 181)	12
4.1 Learning Objectives	12
4.2 What is JSR 181?	12
4.3 Full Annotation Reference	12
4.4 @SOAPBinding in Depth	12
4.5 Handler Chains	13
4.6 The wsdlLocation Attribute	14
4.7 Review Questions	14
5. WSDL to Java and Java to WSDL Paths	15

5.1 Learning Objectives	15
5.2 The Two Development Paths	15
5.3 WSDL-First: wsimport	15
5.4 Using wsimport-Generated Code	16
5.5 Java-First: wsgen	16
5.6 JAXB Binding Customisation	16
5.7 Review Questions	17
6. Provider and Dispatch APIs	18
6.1 Learning Objectives	18
6.2 Why Low-Level APIs?	18
6.3 The Provider<T> API — Service Side	18
6.4 Service.Mode Options	19
6.5 The Dispatch<T> API — Client Side	19
6.6 Asynchronous Dispatch	19
6.7 Review Questions	20
7. SAAJ and JAXP	21
7.1 Learning Objectives	21
7.2 SAAJ — SOAP with Attachments API for Java	21
SAAJ Object Hierarchy	21
7.3 Building a SOAP Message with SAAJ	21
7.4 JAXP — Java API for XML Processing	22
7.5 StAX — Streaming API for XML	22
7.6 Review Questions	23
8. JAX-RS for RESTful Services	24
8.1 Learning Objectives	24
8.2 JAX-RS Overview	24
8.3 Core JAX-RS Annotations	24
8.4 A Complete Resource Example	24
8.5 Application Deployment	25
8.6 Exception Mapping	26
8.7 JAX-RS Filters	26
8.8 Review Questions	26
9. JAXR — Java API for XML Registries	27
9.1 Learning Objectives	27
9.2 What is JAXR?	27
9.3 JAXR Architecture	27
9.4 JAXR Object Model	28
9.5 Connecting to a UDDI Registry	28
9.6 Querying the Registry	28

9.7 Publishing to the Registry	29
9.8 Modern Alternatives to JAXR.....	29
9.9 Review Questions	30
Appendix A — Quick Reference	30
Key APIs Summary	30

1. Scenarios for Hosting Web Services

1.1 Learning Objectives

- Identify common enterprise scenarios that require hosting web services
- Understand deployment topologies: standalone, servlet container, Java EE container
- Appreciate the role of Java EE in providing managed web service infrastructure

1.2 Why Host Web Services in Java EE?

Java EE (now Jakarta EE) provides a managed runtime that offers services that web service hosts need out of the box: thread pooling, transaction management, security, lifecycle management, and dependency injection. Hosting web services in a Java EE container means the service developer focuses on business logic rather than infrastructure.

Key Insight

A web service endpoint is just a managed component — a POJO annotated with `@WebService` or a Servlet wired to a JAX-WS implementation. The container handles deployment, thread management, and integration with container services like JTA, JPA, and EJB.

1.3 Common Hosting Scenarios

Scenario 1 — Business-to-Business (B2B) Integration

Partners need to exchange structured data (orders, invoices, shipping notifications) across organisational boundaries. Java EE containers provide the robustness and scalability required for high-volume, mission-critical B2B traffic.

- Typical stack: EJB endpoint + WS-Security + JMS for async processing
- Example: EDI replacement — purchase orders delivered as SOAP messages over HTTPS

Scenario 2 — Internal Service Decomposition (SOA)

Large monolithic applications are decomposed into services consumed by multiple front-ends (web, mobile, portal). Java EE makes each service a deployable WAR or EJB JAR with well-defined contracts.

- Typical stack: Servlet endpoint or `@Stateless` EJB as `@WebService`
- Example: A customer management EJB exposed as a SOAP service consumed by a portal and a batch process

Scenario 3 — Legacy System Wrapping

Existing COBOL, C++, or proprietary systems gain a modern web service facade without rewriting the core logic.

- Java EE adapter pattern: JAX-WS endpoint delegates to JCA resource adapter
- Example: Mainframe CICS transaction wrapped as a SOAP service via JCA

Scenario 4 — RESTful Resource Exposure

Mobile and browser clients need lightweight JSON/XML APIs. JAX-RS in Java EE provides annotation-driven REST endpoints deployable as part of the same application.

- Typical stack: JAX-RS resource class deployed in a WAR alongside EJBs
- Example: Product catalogue exposed as a REST API for a mobile app

1.4 Deployment Topologies

Topology	Container	Best For	Example
Standalone	SE + Endpoint.publish()	Development / testing	Endpoint.publish("http://localhost:8080/", new MyService())
Servlet Container	Tomcat / Jetty	Lightweight SOAP/REST	JAX-WS + Tomcat via metro or CXF
EJB Container	GlassFish / WildFly	Full Java EE, transactions, security	@Stateless + @WebService
Managed Cluster	WebSphere / WebLogic	Enterprise HA, load-balancing	Cluster-aware JAX-WS deployment

1.5 Container-Provided Services

When a web service is hosted in a Java EE container, the following services become automatically available:

- Security — container-managed authentication and authorisation via @RolesAllowed, JAAS
- Transactions — JTA-managed transactions via @TransactionAttribute on EJB endpoints
- Concurrency — container manages thread pools; developers write single-threaded code
- Lifecycle — @PostConstruct / @PreDestroy lifecycle callbacks
- Dependency Injection — @Resource, @EJB, @Inject available in endpoint classes
- Pooling — EJB container pools endpoint instances for high throughput

1.6 Review Questions

1. Name three Java EE container services that benefit a hosted web service endpoint.
2. What is the difference between hosting a web service in a Servlet container vs an EJB container?
3. Describe a legacy wrapping scenario and explain which Java EE technologies would be involved.

2. Web Services in Java EE

2.1 Learning Objectives

- Understand how Java EE integrates web service technologies
- Map the Java EE web service APIs to the standards they implement
- Understand the role of the Web Services for Java EE (JSR 109) specification

2.2 The Java EE Web Services Stack

Java EE provides a comprehensive, layered set of APIs for building and consuming web services. Each API addresses a distinct layer of the web service stack:

Term	Definition
JAX-WS 2.x	Java API for XML-Based Web Services. Core SOAP endpoint and client API.
JAXB 2.x	Java Architecture for XML Binding. Maps Java objects to/from XML schemas.
SAAJ 1.3	SOAP with Attachments API for Java. Low-level SOAP message manipulation.
JAX-RS 2.x	Java API for RESTful Web Services. Annotation-driven REST resource model.
JAXR 1.0	Java API for XML Registries. Client API for UDDI and ebXML registries.
JAXP	Java API for XML Processing. SAX, DOM, and StAX parsers; XSLT transforms.
JSR 109	Web Services for Java EE. Defines deployment model and container integration.
WS-Metadata	JSR 181. Defines <code>@WebService</code> and companion annotations for POJO endpoints.

2.3 JSR 109 — Web Services for Java EE

JSR 109 is the foundational specification that defines how web services are packaged, deployed, and managed within a Java EE container. Key contributions:

- Defines the `wsdl:service` deployment descriptor elements in `web.xml` and `ejb-jar.xml`
- Specifies how a WSDL is associated with a Java EE endpoint at deployment time
- Defines the Port Component model — a named endpoint unit within a deployment
- Mandates that JAX-WS endpoints be manageable by the container lifecycle
- Specifies handler chains and their deployment descriptor syntax

Port Component

A Port Component is the unit of deployment for a web service in Java EE. It binds:

- A service endpoint interface (SEI) — the Java contract
- A service implementation bean (SIB) — the implementing class
- A WSDL port — the concrete binding and network address
- A handler chain (optional) — for cross-cutting message processing

2.4 Two Endpoint Types

Endpoint Type	Base Class	Annotation	Container Services	Use When
JAX-WS Servlet	HttpServlet (impl'd by runtime)	@WebService on POJO	Limited (no EJB)	Servlet container only; no transactions needed
JAX-WS EJB	@Stateless or @Singleton EJB	@WebService + @Stateless	Full (JTA, JNDI, @Inject)	Transactions, pooling, security, DI needed

2.5 Web Service Client Models

Java EE supports two client programming models for invoking web services:

- Generated Stub Client — wsimport generates static stubs from WSDL; compile-time safety
- Dynamic Proxy Client — javax.xml.ws.Service.getPort() returns a dynamic proxy at runtime
- Dispatch Client — low-level XML dispatch for cases where no WSDL is available at compile time

2.6 Review Questions

4. Which Java EE specification defines the deployment model for web services?
5. What are the four components that make up a Port Component?
6. When would you choose an EJB endpoint over a Servlet endpoint?

3. JAX-WS and JAXB

3.1 Learning Objectives

- Understand the JAX-WS programming model for SOAP service endpoints and clients
- Understand JAXB's role in marshalling/unmarshalling Java-to-XML
- Write, annotate, and deploy a basic JAX-WS service and client

3.2 JAX-WS Overview

JAX-WS (Java API for XML-Based Web Services) is the primary Java API for building SOAP-based web services. It superseded JAX-RPC (JSR 101). JAX-WS:

- Uses annotations (JSR 181) to turn a plain Java class into a web service
- Delegates XML binding to JAXB 2.x (replacing the fragile value type encoding of JAX-RPC)
- Supports both document/literal/wrapped (recommended) and RPC/literal styles
- Provides both synchronous and asynchronous client invocation patterns
- Integrates with WS-* via the handler chain and binding extension model

3.3 A Minimal JAX-WS Service

```
import javax.xml.ws.WebService;  
import javax.xml.ws.WebMethod;  
import javax.xml.ws.WebParam;  
  
@WebService(                                     // Marks class as SOAP  
    endpoint  
        serviceName      = "CustomerService",  
        portName         = "CustomerPort",  
        targetNamespace = "http://example.com/customer"  
    )  
public class CustomerServiceImpl {  
  
    @WebMethod(operationName = "GetCustomer") // Exposed as WSDL operation  
    public Customer getCustomer(  
        @WebParam(name = "customerId") String id) {  
        // business logic...  
        return customerRepository.findById(id);  
    }  
  
    @WebMethod(exclude = true) // Not exposed  
    public void internalHelper() { ... }  
}
```

3.4 Key JAX-WS Annotations

Term	Definition
@WebService	Marks a class as a web service endpoint; sets service/port name and namespace.

@WebMethod	Marks a method as a WSDL operation; can set operationName or exclude=true.
@WebParam	Controls the XML element name and mode (IN/OUT/INOUT) of a parameter.
@WebResult	Controls the XML element name of the return value.
@SOAPBinding	Specifies the SOAP binding style (DOCUMENT/RPC) and use (LITERAL/ENCODED).
@HandlerChain	Associates a handler chain XML file with the endpoint or client.
@Oneway	Marks an operation as one-way (no response expected); return type must be void.
@RequestWrapper	Specifies wrapper class for the doc/literal/wrapped request element.
@ResponseWrapper	Specifies wrapper class for the doc/literal/wrapped response element.

3.5 JAXB — Java Architecture for XML Binding

JAXB provides the mechanism by which Java objects are serialised to XML (marshalling) and XML is deserialised back to Java objects (unmarshalling). JAX-WS delegates all type mapping to JAXB.

JAXB Annotations

Term	Definition
@XmlRootElement	Marks a class as a top-level XML element; defines the element name and namespace.
@XmlElement	Maps a field or property to an XML element; controls name, required, nillable.
@XmlAttribute	Maps a field to an XML attribute.
@XmlType	Defines the XML Schema complexType mapping; controls element order (propOrder).
@XmlEnum	Marks a Java enum for XML enum type mapping.
@XmlTransient	Excludes a field from XML mapping.
@XmlAccessorType	Controls whether JAXB uses fields or properties for access.

```
@XmlRootElement(name = "customer",
    namespace = "http://example.com/customer")
@XmlType(propOrder = { "id", "name", "tier", "creditLimit" })
public class Customer {

    @XmlElement(required = true)
    private String id;

    @XmlElement(required = true)
    private String name;

    @XmlElement(defaultValue = "STANDARD")
```

```
private String tier;

@XmlAttribute(name = "currency")
private String currency = "GBP";

@XmlElement
private BigDecimal creditLimit;

// getters and setters...
}
```

3.6 Marshalling and Unmarshalling

```
// Marshal Java → XML
JAXBContext ctx = JAXBContext.newInstance(Customer.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(customer, System.out);

// Unmarshal XML → Java
Unmarshaller u = ctx.createUnmarshaller();
Customer c = (Customer) u.unmarshal(new File("customer.xml"));

// Schema validation during unmarshalling
SchemaFactory sf =
SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);
Schema schema = sf.newSchema(new File("customer.xsd"));
u.setSchema(schema);
```

3.7 Review Questions

7. What is the difference between `@WebService` on an interface vs on a class?
8. Write the JAXB annotations needed to map a Java class 'Invoice' to an XML element `<invoice>` with namespace `http://example.com/billing`.
9. What does `@Oneway` imply about the HTTP response and the return type of the annotated method?

4. Web Services Metadata (JSR 181)

4.1 Learning Objectives

- Understand the purpose and scope of JSR 181
- Apply metadata annotations to configure SOAP message format, WSDL mapping, and handler chains
- Understand the relationship between annotations and generated WSDL

4.2 What is JSR 181?

JSR 181 (Web Services Metadata for the Java Platform) defines the core annotations that turn plain Java classes (POJOs or EJBs) into web service endpoints without requiring separate deployment descriptor files. JSR 181 annotations are:

- Processed at deployment time by the Java EE container or a standalone JAX-WS runtime
- Used to derive the WSDL automatically (Java-first path) when no WSDL is pre-supplied
- Integrated with JAX-WS 2.x — JSR 181 annotations complement JAX-WS runtime annotations

4.3 Full Annotation Reference

Term	Definition
@WebService	Core annotation. serviceName, portName, endpointInterface, targetNamespace, wsdlLocation.
@WebMethod	operationName (WSDL op name), action (SOAPAction), exclude (hide from WSDL).
@OneWay	Marks operation as one-way. Must return void. No response WSDL output message.
@WebParam	name, targetNamespace, mode (IN/OUT/INOUT), header (true = SOAP header part).
@WebResult	name, targetNamespace, header, partName (for message part name in WSDL).
@SOAPBinding	style (DOCUMENT/RPC), use (LITERAL/ENCODED), parameterStyle (WRAPPED/BARE).
@HandlerChain	file attribute: path to handler chain XML descriptor. Applied to class or method.
@InitParam	Name/value pair passed to Handler.init(). Used within @HandlerChain descriptor.

4.4 @SOAPBinding in Depth

The @SOAPBinding annotation controls how the JAX-WS runtime generates SOAP messages and the corresponding WSDL binding. For interoperability, always use:

```
@WebService
```

```

@SOAPBinding(
    style          = SOAPBinding.Style.DOCUMENT,
    use            = SOAPBinding.Use.LITERAL,
    parameterStyle = SOAPBinding.ParameterStyle.WRAPPED // WS-I compliant
)
public class OrderService { ... }

```

Combination	WS-I Compliant	Recommended	Notes
DOCUMENT / LITERAL / WRAPPED	Yes	✓ Best	Default when @SOAPBinding omitted with JAX-WS
DOCUMENT / LITERAL / BARE	Yes	Usable	Each param is a separate top-level element
RPC / LITERAL	Yes	Avoid	Less interoperable; no schema for body
RPC / ENCODED	No	Never	Legacy only; violates WS-I Basic Profile

4.5 Handler Chains

Handler chains provide interceptor-style processing of SOAP messages — used for logging, authentication, auditing, and transformation. Two types:

- **SOAPHandler** — access to the full SOAP message as a `SOAPMessage` object
- **LogicalHandler** — access only to the logical message payload (protocol-independent)

```

// Handler implementation
public class LoggingHandler implements SOAPHandler<SOAPMessageContext> {

    public boolean handleMessage(SOAPMessageContext ctx) {
        Boolean outbound = (Boolean)
ctx.get(MessageContext.MESSAGE_OUTBOUND_PROPERTY);
        SOAPMessage msg = ctx.getMessage();
        msg.writeTo(System.out);           // log message
        return true;                       // continue chain
    }

    public boolean handleFault(SOAPMessageContext ctx) {
        return true;
    }

    public Set<QName> getHeaders() { return Collections.emptySet(); }
    public void close(MessageContext ctx) {}
}

```

Handler chain XML file (handler-chain.xml):

```

<?xml version="1.0" encoding="UTF-8"?>
<handler-chains xmlns="http://java.sun.com/xml/ns/javaee">
  <handler-chain>
    <handler>
      <handler-class>com.example.LoggingHandler</handler-class>
    </handler>
  </handler-chain>
</handler-chains>

```

```
<handler>
  <handler-class>com.example.SecurityHandler</handler-class>
</handler>
</handler-chain>
</handler-chains>
```

4.6 The wsdlLocation Attribute

When `@WebService(wsdlLocation="META-INF/wsdl/CustomerService.wsdl")` is specified, the container uses the pre-existing WSDL as the authoritative contract. The Java class must implement the interface the WSDL describes. This is the WSDL-first approach.

Best Practice

For external-facing services with fixed contracts, use `wsdlLocation` (WSDL-first).
For internal services where the Java code is the authoritative design, omit `wsdlLocation` (Java-first).
Never mix the approaches within the same service — it causes WSDL inconsistencies.

4.7 Review Questions

10. What is the purpose of `@WebParam(header=true)`?
11. Explain the difference between a `SOAPHandler` and a `LogicalHandler`. When would you use each?
12. What does `wsdlLocation` on `@WebService` tell the runtime?

5. WSDL to Java and Java to WSDL Paths

5.1 Learning Objectives

- Understand and apply both the WSDL-first and Java-first development approaches
- Use the `wsimport` tool to generate Java artifacts from a WSDL
- Use the `wsgen` tool to generate WSDL and XSD from annotated Java
- Choose the appropriate path for a given scenario

5.2 The Two Development Paths

Aspect	WSDL-First (Contract-First)	Java-First (Code-First)
Starting Point	WSDL document	Annotated Java class
Tool	<code>wsimport</code> → generates Java	<code>wsgen</code> → generates WSDL/XSD
Contract stability	Contract defined before implementation	Contract derived from code
Interoperability	Highest (WSDL is authoritative)	Good if using doc/lit/wrapped
Best for	External partners; multiple implementations	Rapid prototyping; internal services
Pitfall	More upfront design effort	Annotation mistakes can break interop

5.3 WSDL-First: `wsimport`

`wsimport` reads a WSDL and generates the following Java artifacts:

- Service class — extends `javax.xml.ws.Service`; factory for port proxies
- SEI (Service Endpoint Interface) — Java interface mapping each WSDL operation to a Java method
- JAXB-annotated value types — one class per WSDL schema `complexType`
- Wrapper classes — request and response wrapper beans for doc/lit/wrapped operations
- Exception classes — one per `wsdl:fault` element

```
# Basic usage
wsimport -keep -verbose \
  -p com.example.customer.client \
  -d target/generated-sources \
  http://api.example.com/CustomerService?wsdl

# With custom binding (override generated class names)
wsimport -keep -b bindings.xml \
  -p com.example.customer.client \
  CustomerService.wsdl

# Useful flags:
# -keep      Keep generated source files
# -p        Java package for generated classes
```

```
# -b          External JAXB/JAX-WS binding customisation file
# -extension  Allow vendor extensions
# -wsdllocation Override the @WebServiceClient wsdlLocation URI
```

5.4 Using wsimport-Generated Code

```
// 1. Create the Service object (reads WSDL at construction or from
annotation)
CustomerService service = new CustomerService();

// 2. Get a port proxy - implements the SEI
CustomerPortType port = service.getCustomerPort();

// 3. Invoke the operation - marshalling/unmarshalling handled by JAX-WS +
JAXB
GetCustomerRequest req = new GetCustomerRequest();
req.setCustomerId("CUST-001");

GetCustomerResponse resp = port.getCustomer(req);
System.out.println(resp.getCustomer().getName());

// 4. Configuring the endpoint address at runtime
BindingProvider bp = (BindingProvider) port;
bp.getRequestContext().put(
    BindingProvider.ENDPOINT_ADDRESS_PROPERTY,
    "https://api.example.com/CustomerService");
```

5.5 Java-First: wsgen

wsgen reads compiled JAX-WS annotated classes and generates the WSDL and XSD artifacts needed for deployment. It must be run after compilation.

```
# Generate WSDL and XSD from annotated class
wsgen -keep -verbose \
    -cp target/classes \
    -wsdl \
    -d target/generated \
    com.example.customer.CustomerServiceImpl

# Useful flags:
# -wsdl      Generate WSDL document
# -r        Resource directory for generated WSDL/XSD
# -s        Source directory for generated Java source
# -cp       Classpath containing the endpoint class
```

wsgen also generates the wrapper classes required for doc/lit/wrapped operations (if JAXB cannot derive them from the parameter types alone).

5.6 JAXB Binding Customisation

When wsimport generates code from a WSDL, the default class names may conflict with existing code or be too generic. A JAXB binding file allows customisation:


```
<!-- bindings.xml -->
<bindings xmlns="http://java.sun.com/xml/ns/jaxb" version="2.1"
  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <bindings schemaLocation="CustomerService.xsd"
node="//xs:complexType[@name='CustomerType']">
    <class name="CustomerInfo"/>        <!-- rename generated class -->
  </bindings>
  <bindings schemaLocation="CustomerService.xsd"
node="//xs:element[@name='tier']">
    <property name="customerTier"/>    <!-- rename generated property -->
  </bindings>
</bindings>
```

5.7 Review Questions

13. List four Java artifacts generated by wsimport from a WSDL.
14. When must wsgen be run relative to the Java compilation step?
15. Explain the purpose of a JAXB binding customisation file.
16. You receive a WSDL from a trading partner. Which development path should you use and why?

6. Provider and Dispatch APIs

6.1 Learning Objectives

- Understand when and why to use the Provider and Dispatch APIs over generated stubs
- Implement a `Provider<SOAPMessage>` service endpoint
- Use the Dispatch API for dynamic client invocations

6.2 Why Low-Level APIs?

Generated stubs and SEIs cover the majority of use cases. However, there are scenarios where you need direct access to the raw SOAP message:

- Generic routing or mediation components that must handle any message shape
- Testing harnesses that validate raw SOAP traffic
- Legacy bridging where the WSDL is unknown or changes frequently
- High-performance scenarios where JAXB marshalling overhead must be avoided
- Protocol-level debugging and monitoring tools

6.3 The `Provider<T>` API — Service Side

`Provider<T>` is the dynamic service endpoint interface. A class implementing `Provider<T>` receives the raw incoming message and must return the raw response. The generic type `T` determines the message representation:

Term	Definition
<code>Provider<SOAPMessage></code>	Full SOAP envelope access via SAAJ <code>SOAPMessage</code> . Use with <code>mode=MESSAGE</code> .
<code>Provider<Source></code>	XML payload as <code>javax.xml.transform.Source</code> . Use with <code>mode=PAYLOAD</code> (body only).
<code>Provider<DataSource></code>	Binary MIME attachment access. Use for MTOM/SwA scenarios.

```
@WebServiceProvider(
    serviceName      = "EchoService",
    portName         = "EchoPort",
    targetNamespace  = "http://example.com/echo",
    wsdlLocation     = "META-INF/wsdl/EchoService.wsdl"
)
@ServiceMode(value = Service.Mode.MESSAGE) // Receive full SOAP envelope
public class EchoProvider implements Provider<SOAPMessage> {

    @Override
    public SOAPMessage invoke(SOAPMessage request) {
        try {
            // Inspect or transform the raw SOAP message
            request.writeTo(System.out);

            // Build a response SOAP message
            MessageFactory mf = MessageFactory.newInstance();
            SOAPMessage response = mf.createMessage();
            SOAPBody body = response.getSOAPBody();
        } catch (Exception e) {
            // Handle exception
        }
    }
}
```

```

        body.addBodyElement(
            body.createQName("EchoResponse", "ns"))
            .addTextNode("Echo OK");
        return response;
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
}

```

6.4 Service.Mode Options

Mode	What Provider<T> Receives	Typical T	Use When
PAYLOAD	XML inside soap:Body only	Source	You only care about the business payload
MESSAGE	Entire SOAP Envelope	SOAPMessage	You need to read/write SOAP headers

6.5 The Dispatch<T> API — Client Side

Dispatch<T> is the dynamic client API, the counterpart to Provider<T>. Instead of using a generated stub (port proxy), you construct and send messages programmatically. Useful when the WSDL is not available at compile time, or when you need to send arbitrary XML.

```

// Create the Service object manually (no wsimport needed)
QName serviceName = new QName("http://example.com/echo", "EchoService");
QName portName    = new QName("http://example.com/echo", "EchoPort");

Service service = Service.create(new URL(wsdlUrl), serviceName);

// Create a Dispatch for full SOAP message (MESSAGE mode)
Dispatch<SOAPMessage> dispatch = service.createDispatch(
    portName,
    SOAPMessage.class,
    Service.Mode.MESSAGE);

// Build the request SOAP message
MessageFactory mf = MessageFactory.newInstance();
SOAPMessage request = mf.createMessage();
SOAPBody body = request.getSOAPBody();
body.addBodyElement(body.createQName("EchoRequest", "ns"))
    .addTextNode("Hello from Dispatch");

// Invoke — synchronous
SOAPMessage response = dispatch.invoke(request);
response.writeTo(System.out);

```

6.6 Asynchronous Dispatch

Dispatch also supports asynchronous invocation — useful for long-running operations or when the client cannot block:

```
// Polling model
Response<SOAPMessage> future = dispatch.invokeAsync(request);
// ... do other work ...
SOAPMessage response = future.get(); // blocks until response arrives

// Callback model
dispatch.invokeAsync(request, new AsyncHandler<SOAPMessage>() {
    public void handleResponse(Response<SOAPMessage> res) {
        try {
            SOAPMessage response = res.get();
            // process response
        } catch (Exception e) {
            // handle fault
        }
    }
});
```

6.7 Review Questions

17. What is the difference between `Service.Mode.MESSAGE` and `Service.Mode.PAYLOAD`?
18. Name three scenarios where `Dispatch` is preferable to a generated stub client.
19. Implement a `Provider<Source>` that reads the incoming XML payload and returns a static XML response.

7. SAAJ and JAXP

7.1 Learning Objectives

- Understand the SAAJ API for constructing and reading SOAP messages
- Use SAAJ to add SOAP headers, body elements, and attachments
- Understand JAXP parsers (SAX, DOM, StAX) and transforms (XSLT)
- Know when to use SAAJ vs JAXP in a web service context

7.2 SAAJ — SOAP with Attachments API for Java

SAAJ provides a Java object model for building and reading SOAP messages at the message level. It is used internally by JAX-WS but can also be used directly by applications through `SOAPHandler` or `Dispatch`.

SAAJ Object Hierarchy

Term	Definition
SOAPMessage	Top-level container. Contains MimeHeaders, SOAPPart, and Attachments.
SOAPPart	The XML content of the message. Contains the SOAPEnvelope.
SOAPEnvelope	Root XML element of the SOAP message. Contains SOAPHeader and SOAPBody.
SOAPHeader	Container for header blocks. Access via <code>getSOAPHeader()</code> .
SOAPHeaderElement	Individual header block with namespace, mustUnderstand, role/actor attributes.
SOAPBody	Container for the business payload or a SOAPFault.
SOAPBodyElement	XML element inside the Body.
SOAPFault	Represents a SOAP fault: faultcode, faultstring, faultactor, detail.
AttachmentPart	Non-XML MIME attachment (binary file, image, etc.).

7.3 Building a SOAP Message with SAAJ

```
MessageFactory mf =
MessageFactory.newInstance(SOAPConstants.SOAP_1_2_PROTOCOL);
SOAPMessage msg = mf.createMessage();
SOAPEnvelope env = msg.getSOAPPart().getEnvelope();

// Add a custom namespace
env.addNamespaceDeclaration("cust", "http://example.com/customer");

// Build Header
SOAPHeader header = env.getHeader();
SOAPHeaderElement authToken = header.addHeaderElement(
    env.createQName("AuthToken", "cust"));
```

```

authToken.setMustUnderstand(true);
authToken.addTextNode("Bearer eyJhbGci...");

// Build Body
SOAPBody body = env.getBody();
SOAPBodyElement getCustomer = body.addBodyElement(
    env.createQName("GetCustomerRequest", "cust"));
getCustomer.addChildElement(
    env.createQName("customerId", "cust")).addTextNode("CUST-001");

// Add attachment
AttachmentPart att = msg.createAttachmentPart();
att.setContent(imageBytes, "image/png");
att.setContentId("<logo@example.com>");
msg.addAttachmentPart(att);

msg.saveChanges();
msg.writeTo(System.out);

```

7.4 JAXP — Java API for XML Processing

JAXP provides Java's standard API for XML parsing and transformation. It is used by JAX-WS, JAXB, and SAAJ underneath, but can be used directly when fine-grained control over XML processing is needed.

Parser/API	Type	Memory Model	Best For	Key Class
SAX	Event-driven	Streaming (low)	Large docs; no random access	SAXParserFactory
DOM	Tree-based	In-memory tree	Random access; edit & write back	DocumentBuilderFactory
StAX	Pull-based	Streaming (low)	Better control than SAX; high perf	XMLInputFactory
XSLT	Transformation	Depends on processor	Transform XML to XML/HTML/text	TransformerFactory

7.5 StAX — Streaming API for XML

StAX is the preferred parser for high-performance, streaming XML processing in Java EE web services. It provides a cursor-based pull model:

```

XMLInputFactory factory = XMLInputFactory.newInstance();
// Security: disable external entity processing
factory.setProperty(XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false);
factory.setProperty(XMLInputFactory.SUPPORT_DTD, false);

XMLStreamReader reader = factory.createXMLStreamReader(
    new FileInputStream("order.xml"));

while (reader.hasNext()) {
    int event = reader.next();
    if (event == XMLStreamConstants.START_ELEMENT) {

```

```
String localName = reader.getLocalName();
if ("orderId".equals(localName)) {
    System.out.println("Order ID: " + reader.getElementText());
}
}
}
reader.close();
```

Security Alert — Disable External Entities

Always set `IS_SUPPORTING_EXTERNAL_ENTITIES=false` and `SUPPORT_DTD=false` on `XMLInputFactory`.

Failure to do so exposes the application to XXE (XML External Entity) injection attacks, which can read arbitrary files on the server or trigger SSRF requests.

7.6 Review Questions

20. What is the difference between `SOAPHeader` and `SOAPHeaderElement` in SAAJ?
21. When would you use StAX over DOM for parsing XML in a JAX-WS context?
22. How do you add a binary attachment to a SOAP message using SAAJ?
23. What security property must be set on `XMLInputFactory` to prevent XXE attacks?

8. JAX-RS for RESTful Services

8.1 Learning Objectives

- Understand the JAX-RS programming model for REST resource classes
- Apply JAX-RS annotations to build a fully functional REST API
- Handle content negotiation, exception mapping, and filters in JAX-RS
- Deploy a JAX-RS application within a Java EE container

8.2 JAX-RS Overview

JAX-RS (Java API for RESTful Web Services, JSR 311/339/370) provides a Java annotation-based framework for building RESTful web services. It is the Java EE counterpart to JAX-WS for REST. Key design goals:

- Annotation-driven — minimal boilerplate; no framework-specific base classes required
- POJO-based — resource classes are plain Java objects managed by the JAX-RS runtime
- Content negotiation — `@Produces` and `@Consumes` annotations declare media types
- Provider model — `MessageBodyReader/Writer` for custom serialisation, `ExceptionHandler` for errors
- Filter and Interceptor API — for cross-cutting concerns (auth, logging, CORS)

8.3 Core JAX-RS Annotations

Term	Definition
<code>@Path</code>	Binds a URI path template to a resource class or method. Supports {param} templates.
<code>@GET/@POST/etc.</code>	HTTP method binding annotations. Maps a method to an HTTP verb.
<code>@Produces</code>	Declares the media types a method or class can produce (response Content-Type).
<code>@Consumes</code>	Declares the media types a method or class can consume (request Content-Type).
<code>@PathParam</code>	Injects a URI template variable into a method parameter.
<code>@QueryParam</code>	Injects a URI query parameter (?key=value) into a method parameter.
<code>@HeaderParam</code>	Injects an HTTP request header value into a method parameter.
<code>@FormParam</code>	Injects a form field from an application/x-www-form-urlencoded body.
<code>@BeanParam</code>	Aggregates multiple <code>@Param</code> annotations into a single bean class.
<code>@Context</code>	Injects JAX-RS context objects: <code>UriInfo</code> , <code>HttpHeaders</code> , <code>SecurityContext</code> , <code>Request</code> .
<code>@ApplicationPath</code>	Defines the base URI for all resources in the JAX-RS Application class.

8.4 A Complete Resource Example


```

@Path("/customers")
@Produces(MediaType.APPLICATION_XML)
@Consumes(MediaType.APPLICATION_XML)
public class CustomerResource {

    @Inject
    private CustomerService svc;          // CDI injection works in Java EE

    // GET /customers - list all
    @GET
    public List<Customer> list(
        @QueryParam("tier") String tier,
        @QueryParam("page") @DefaultValue("0") int page) {
        return svc.find(tier, page);
    }

    // GET /customers/{id}
    @GET
    @Path("{id}")
    public Response get(@PathParam("id") String id) {
        Customer c = svc.findById(id);
        if (c == null) throw new NotFoundException("Customer not found: " +
id);
        return Response.ok(c).build();
    }

    // POST /customers - create
    @POST
    public Response create(Customer customer, @Context UriInfo ui) {
        Customer created = svc.create(customer);
        URI location = ui.getAbsolutePathBuilder()
            .path(created.getId()).build();
        return Response.created(location).entity(created).build(); // 201
    }

    // PUT /customers/{id}
    @PUT
    @Path("{id}")
    public Customer update(
        @PathParam("id") String id, Customer customer) {
        return svc.update(id, customer);
    }

    // DELETE /customers/{id}
    @DELETE
    @Path("{id}")
    public void delete(@PathParam("id") String id) {
        svc.delete(id);          // returns 204 No Content
    }
}

```

8.5 Application Deployment

A JAX-RS application is defined by a class extending `javax.ws.rs.core.Application` annotated with `@ApplicationPath`:

```

@ApplicationPath("/api")          // All resources under /api/*
public class MyApplication extends Application {
    // Empty body - container scans for @Path classes automatically
    // Alternatively, override getClasses() or getSingletons() to register
    manually
}

```

```
}
```

8.6 Exception Mapping

ExceptionHandler converts exceptions to appropriate HTTP responses — the JAX-RS equivalent of exception handlers:

```
@Provider
public class ValidationExceptionHandler
    implements ExceptionMapper<ConstraintViolationException> {

    public Response toResponse(ConstraintViolationException ex) {
        Map<String,String> errors = new LinkedHashMap<>();
        ex.getConstraintViolations().forEach(cv ->
            errors.put(cv.getPropertyPath().toString(), cv.getMessage()));
        return Response
            .status(Response.Status.BAD_REQUEST)    // 400
            .entity(errors)
            .type(MediaType.TEXT_PLAIN)
            .build();
    }
}
```

8.7 JAX-RS Filters

Filters intercept requests and responses for cross-cutting concerns. They implement ContainerRequestFilter or ContainerResponseFilter:

```
@Provider
@Priority(Priorities.AUTHENTICATION)
public class JwtAuthFilter implements ContainerRequestFilter {

    public void filter(ContainerRequestContext ctx) {
        String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);
        if (auth == null || !auth.startsWith("Bearer ")) {
            ctx.abortWith(Response.status(401).build());
            return;
        }
        String token = auth.substring(7);
        // validate JWT...
    }
}
```

8.8 Review Questions

24. What is the difference between @Produces and @Consumes?
25. How do you return a 201 Created response with a Location header from a POST method?
26. Describe the JAX-RS ExceptionMapper mechanism and give an example of when to use it.
27. What annotation is used to deploy a JAX-RS application and set the base URI path?

9. JAXR — Java API for XML Registries

9.1 Learning Objectives

- Understand the purpose and scope of JAXR
- Understand the JAXR object model and registry concepts
- Perform basic JAXR operations: publish, find, and bind
- Appreciate the current state of JAXR and its practical alternatives

9.2 What is JAXR?

JAXR (Java API for XML Registries, JSR 93) is a Java API for interacting with XML-based service registries. It provides a unified programming model for two registry specifications:

- UDDI (Universal Description, Discovery and Integration) v2 and v3
- ebXML Registry / Repository specification

JAXR abstracts over the underlying registry protocol, allowing Java code to publish, query, and bind to services without being tied to UDDI or ebXML specifics.

Historical Context

JAXR was designed when UDDI was expected to become the universal service registry. After the major public UDDI registries shut down in 2006, JAXR's importance declined significantly.

JAXR was removed from the Java EE 7 platform (deprecated) and is absent from Jakarta EE.

Understanding JAXR is still valuable for maintaining legacy enterprise applications.

9.3 JAXR Architecture

Term	Definition
JAXR Client	Application code that uses the javax.xml.registry API.
ConnectionFactory	javax.xml.registry.ConnectionFactory. Obtained via JNDI; creates Connections.
Connection	Represents a session with the registry. Created from ConnectionFactory.
RegistryService	Entry point for registry operations. Obtained from Connection.
BusinessQueryMgr	Handles find/search operations: findOrganizations, findServices, findClassificationSchemes.
BusinessLifecycleMgr	Handles create/update/delete: saveOrganizations, saveServices, deleteObjects.
DeclarativeQueryMgr	Handles SQL-like queries for ebXML registries.

9.4 JAXR Object Model

JAXR defines a rich object model that maps to both UDDI and ebXML registry concepts:

Term	Definition
Organization	Represents a business entity (maps to UDDI businessEntity).
Service	Represents a web service offered by an Organization.
ServiceBinding	Technical endpoint information for a Service (maps to UDDI bindingTemplate).
SpecificationLink	Link from a ServiceBinding to a technical specification (e.g., WSDL tModel).
ClassificationScheme	A taxonomy used to categorise objects (NAICS, UNSPSC, etc.).
Classification	Associates an object with a value in a ClassificationScheme.
ExtrinsicObject	Arbitrary document stored in the registry (ebXML-specific).

9.5 Connecting to a UDDI Registry

```
// 1. Obtain the ConnectionFactory (JNDI lookup in Java EE)
ConnectionFactory factory =
    (ConnectionFactory) ctx.lookup("java:comp/env/jaxr/MyRegistry");

// 2. Set properties and create connection
Properties props = new Properties();
props.setProperty("javax.xml.registry.queryManagerURL",
    "http://registry.example.com/uddi/inquiry");
props.setProperty("javax.xml.registry.lifeCycleManagerURL",
    "http://registry.example.com/uddi/publish");
factory.setProperties(props);

Connection conn = factory.createConnection();
conn.setCredentials(Collections.singleton(
    new PasswordAuthentication("user", "pass".toCharArray())));

RegistryService rs = conn.getRegistryService();
BusinessQueryManager bqm = rs.getBusinessQueryManager();
BusinessLifeCycleManager blcm = rs.getBusinessLifeCycleManager();
```

9.6 Querying the Registry

```
// Find organisations by name
Collection<String> namePatterns = Arrays.asList("%Acme%");
BulkResponse response = bqm.findOrganizations(
    null,          // findQualifiers
    namePatterns,  // namePatterns
    null,          // classifications
    null,          // specifications
    null,          // externalIdentifiers
    null           // externalLinks
);

Collection<Organization> orgs = response.getCollection();
for (Organization org : orgs) {
```

```

        System.out.println("Org: " + org.getName().getValue());
        Collection<Service> services = org.getServices();
        for (Service svc : services) {
            System.out.println("  Service: " + svc.getName().getValue());
            Collection<ServiceBinding> bindings = svc.getServiceBindings();
            for (ServiceBinding sb : bindings) {
                System.out.println("    Endpoint: " + sb.getAccessURI());
            }
        }
    }
}

```

9.7 Publishing to the Registry

```

// Create and publish an Organisation with a Service
Organization org = blcm.createOrganization("Acme Corporation");
org.setDescription(blcm.createInternationalString("Order management
services"));

Service svc = blcm.createService("OrderService");
svc.setDescription(blcm.createInternationalString("SOAP order processing
service"));

ServiceBinding sb = blcm.createServiceBinding();
sb.setAccessURI("https://api.example.com/OrderService");

SpecificationLink link = blcm.createSpecificationLink();
link.setSpecificationObject(wsdlTModel); // tModel referencing the WSDL
sb.addSpecificationLink(link);
svc.addServiceBinding(sb);
org.addService(svc);

BulkResponse br = blcm.saveOrganizations(Collections.singleton(org));
if (br.getStatus() == JAXRResponse.STATUS_SUCCESS) {
    System.out.println("Published successfully");
}

```

9.8 Modern Alternatives to JAXR

For new projects, JAXR is not available in Jakarta EE. Alternatives for service discovery include:

Alternative	Type	Key API / Protocol	Use Case
Consul	Service registry	HTTP REST API	Microservices runtime discovery
Eureka (Netflix)	Service registry	REST API	Spring Cloud microservices
Apache jUDDI	UDDI v3 implementation	JAXR or REST client	Legacy UDDI integration
AWS API Gateway	Managed API catalogue	AWS SDK / Console	Cloud-native API management
Backstage (Spotify)	Developer portal	REST API	Internal service catalogues

9.9 Review Questions

28. What two registry standards does JAXR support?
29. Map the JAXR objects Organization, Service, and ServiceBinding to their UDDI equivalents.
30. Why was JAXR removed from Java EE 7?
31. For a new microservices project, which service discovery mechanism would you recommend instead of JAXR, and why?

Appendix A — Quick Reference

Key APIs Summary

API	JSR	Standard	Primary Use
JAX-WS	JSR 224	SOAP web services	Build and consume SOAP endpoints
JAXB	JSR 222	XML binding	Marshal/unmarshal Java ↔ XML
JAX-RS	JSR 370	RESTful services	Build REST resource classes
SAAJ	JSR 67	SOAP messages	Low-level SOAP message construction
JAXP	JAXP 1.4	XML processing	SAX/DOM/StAX parsing; XSLT
JAXR	JSR 93	Registry client	UDDI/ebXML registry access (legacy)
WSMetadata	JSR 181	Annotations	@WebService, @WebMethod etc.
JSR 109	JSR 109	Deployment	Web Services for Java EE container